

Chapter 1 — Scaled Dot-Product Attention

Type: theory + notebook

Ordering: theory-first

Depends on: 0 (conventions)

“Attention is a differentiable, content-based lookup.”

1.1 Motivation

Given a sequence of token representations $\mathbf{x}_1, \dots, \mathbf{x}_n$, we want each token to reach into the sequence and pull information from other tokens it finds relevant — based on content, not position. Scaled dot-product attention is the canonical solution: a differentiable, permutation-equivariant operator that computes each output as a convex combination of value vectors, weighted by query-key similarity.

1.2 Storyline

Attention is a four-step pipeline. Given queries $Q \in \mathbb{R}^{n \times d_k}$, keys $K \in \mathbb{R}^{m \times d_k}$, and values $V \in \mathbb{R}^{m \times d_v}$, each output $\mathbf{o}_i$ is produced by:

1. **Score.** Compute $s_{ij} = \langle \mathbf{q}_i, \mathbf{k}_j \rangle$ — the raw content-match between query i and key j . The matrix $QK^\top \in \mathbb{R}^{n \times m}$ collects all scores.
2. **Scale.** Divide by $\sqrt{d_k}$. This is the variance controller studied in Chapter 3 — for now treat it as a fixed pre-softmax temperature.
3. **Normalize.** Apply row-wise softmax. The row α_i is now a probability distribution over the m source tokens — the attention weights.
4. **Mix.** Output $\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j$ — a convex combination of value rows, weighted by attention.

The whole operation is $\text{softmax}(QK^\top / \sqrt{d_k}) V$. Everything else in the transformer literature — multi-head (Ch. 5), causal masking (Ch. 4), positional encodings (Ch. 6), cross-attention, KV-cache — is a dressing of these four steps.

Why this particular design. Three forces are being balanced: (a) content-addressing (each query decides what to retrieve based on its own state, not its position); (b) differentiability (no argmax — the operation must admit gradients so the projection weights can be learned); (c) permutation equivariance over sources (the operator is invariant to reordering keys/values as long as they are permuted together, which forces positional information to be injected at the input, not in the mechanism). These three pin down the softmax-weighted convex combination uniquely up to parameterization.

Train / val / inference. The operator itself is identical across the three regimes — same four steps, same weights (once trained). Two important wrinkles: - **Attention dropout** (zeroing entries of α , rescaling the rest) is applied during training only. Validation and inference see the undrop'd weights. - **KV-cache at inference.** During autoregressive generation, K and V for past tokens are cached and re-used across steps; only the new query interacts with the full key/value stack. This is an engineering optimization that exploits the causal structure of Ch. 4, not a change to the operator. - **Parallel vs. sequential.** Training runs attention on the full sequence in one shot; inference runs one query per token sequentially (for autoregressive models). The computation is identical, only the batching differs.

1.3 Rigorous core

Definition 1.1 (Scaled dot-product attention). Let $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$, $V \in \mathbb{R}^{m \times d_v}$. Define

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \in \mathbb{R}^{n \times d_v},$$

where softmax is applied row-wise on the m -dimensional rows of the $n \times m$ score matrix.

In self-attention, Q, K, V come from the same input $X \in \mathbb{R}^{n \times d}$ via learned projections $Q = XW_Q, K = XW_K, V = XW_V$ with $W_Q, W_K \in \mathbb{R}^{d \times d_k}$ and $W_V \in \mathbb{R}^{d \times d_v}$.

Proposition 1.2 (Permutation equivariance over sources). For any $m \times m$ permutation matrix P ,

$$\text{Attention}(Q, PK, PV) = \text{Attention}(Q, K, V).$$

Sketch: row-permutation equivariance of softmax plus $P^\top P = I$. (Direct verification.)

Corollary 1.2.1. Self-attention is permutation-equivariant under token reordering. Positional information must therefore be injected before attention (Chapter 6).

Proposition 1.3 (Convex-hull containment). Each output row $\mathbf{o}_i$ lies in the convex hull of $\{\mathbf{v}_1, \dots, \mathbf{v}_m\}$.

Corollary 1.3.1. For every coordinate k , $\min_j V_{jk} \leq (\mathbf{o}_i)_k \leq \max_j V_{jk}$. Attention cannot extrapolate beyond the value range — a sharp, often overlooked constraint.

Proposition 1.4 (Smoothness). $\text{Attention}(Q, K, V)$ is C^∞ in Q, K, V , hence admits gradients everywhere. (Composition of smooth maps; the failure of this property for hard attention is what makes Exercise 1.4 interesting.)

1.4 Numerical example — $d_k = d_v = 2, n = m = 2$ by hand

Let

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix}.$$

Step 1 — score. $QK^\top = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$.

Step 2 — scale. Divide by $\sqrt{2} \approx 1.414$: $\begin{pmatrix} 0.707 & 0.707 \\ 0 & 0.707 \end{pmatrix}$.

Step 3 — softmax row-wise. - Row 1: both entries equal, so softmax gives $(0.5, 0.5)$. - Row 2: $(e^0, e^{0.707})/Z = (1, 2.028)/3.028 \approx (0.330, 0.670)$.

Step 4 — mix with V . - $\mathbf{o}_1 = 0.5 \cdot (10, 0) + 0.5 \cdot (0, 10) = (5, 5)$. - $\mathbf{o}_2 = 0.330 \cdot (10, 0) + 0.670 \cdot (0, 10) \approx (3.30, 6.70)$.

Sanity check against Corollary 1.3.1. Each output coordinate is in $[0, 10]$ — the range of V . ✓

1.5 Mechanics check

- **Score ($QK^\top$).** Measures content similarity between each query and each key via inner product. The Gram-matrix structure is what makes attention “content-based” rather than position-based.

- **Scale** ($/\sqrt{d_k}$). Controls pre-softmax variance; without it, softmax saturates at large d_k (Ch. 3). No effect when d_k is small.
- **Softmax**. Converts scores to a probability distribution. Enforces non-negativity, sum-to-one, differentiability, and order-preservation (Ch. 2). The only step that is non-linear in Q, K .
- **Mix** ($\cdot V$). Produces output as a convex combination of value rows. By Corollary 1.3.1, the mechanism is a contraction in the sense that outputs cannot exceed the value range.
- **Separate projections** W_Q, W_K, W_V . Decouples three roles: what a token asks for (Q), what it advertises as a matching index (K), what it contributes as payload (V). Tying any two collapses expressivity (Exercise 1.4).
- **Row-wise softmax (not column-wise)**. Normalizes across keys per query, not across queries per key. Swapping this would break the “each query pulls from the sequence” semantics; the result would not even have the right units.

1.6 Exercises

1.1 ★★ [Mechanism dissection — hard-lookup limit.] Fix Q, K, V such that for some specific j^* , $s_{ij^*} = \Delta$ and $s_{ij} = 0$ for $j \neq j^*$. Compute $\lim_{\Delta \rightarrow \infty} \mathbf{o}_i$. Now scale the scores by Δ itself: replace s_{ij} by $\Delta \cdot s_{ij}$ (equivalently, scale Q by $\sqrt{\Delta}$). Show that the output converges to $\mathbf{v}_{j^*}$ and quantify the rate: for n keys with a margin-1 winner, how large must Δ be to get $\alpha_{ij^*} > 1 - \epsilon$? This is soft-argmax as the temperature-zero limit, and the result is the bridge to Chapter 2.

1.2 ★★ [Failure-mode construction — tied $W_Q = W_K$.] Suppose $W_Q = W_K$ (tied), so $\mathbf{q}_i = \mathbf{k}_i$ for every i . Show that the score matrix is symmetric: $s_{ij} = s_{ji}$. Construct a concrete $n = 3$ task for which untied W_Q, W_K is strictly more expressive — i.e., the set of attention matrices realizable with tied projections is a proper subset of those realizable with untied projections. What structural property of $QK^\top$ is being given up when we tie? Hint: think about rank and asymmetry.

1.3 ★★★ [Theorem about the operator — universality and limits.] Consider self-attention with fixed W_Q, W_K, W_V as a map $X \in \mathbb{R}^{n \times d} \rightarrow Y \in \mathbb{R}^{n \times d_v}$.

- Prove that Y always lies in the convex hull of $\{W_V \mathbf{x}_1, \dots, W_V \mathbf{x}_n\}$, row-wise. (Corollary 1.3.1.)
- Conclude that a single self-attention layer cannot compute a function whose output range is outside the range of the linear map W_V applied to the input tokens. Give a concrete example of a function from $\mathbb{R}^{n \times 1} \rightarrow \mathbb{R}^{n \times 1}$ that is representable by an MLP but not by any single self-attention layer.
- What does this say about why transformers need both attention and a feedforward block per layer? (You are previewing Chapter 8.)

1.4 ★★★ [Predict-then-verify — soft vs. hard attention.] The notebook will compare three attention variants on a simple retrieval task: (i) standard softmax attention, (ii) softmax with temperature $T \rightarrow 0$ (near-hard attention), and (iii) actual argmax (non-differentiable). The task is: given n keys, one of which matches the query, retrieve the corresponding value.

Predict, before running:

- Which of (i)–(iii) reaches the best retrieval accuracy at convergence, and why?
- Which trains fastest (fewest steps to near-zero loss)? Why?
- Which fails to train at all? Justify from Proposition 1.4 or its failure.
- For small n (say $n = 4$), does (ii) strictly dominate (i)? What about large n (say $n = 1024$)? Give a mechanistic reason for the n -dependence.

State confidence per part (low / medium / high) with one-line reasons. The notebook will test (a)–(d) on a toy copy task.

1.7 Before the notebook (01_attention_by_hand.ipynb)

Write down these predictions before opening the notebook:

1. **Worked-example reproduction.** The numpy implementation should reproduce $(5, 5)$ and $\approx (3.30, 6.70)$ from §1.4 to floating-point precision. Predict the specific floating-point error you expect (order of magnitude).
2. **Perfect-match retrieval.** With $n = 4$ orthogonal keys and a query equal to key 2 scaled by λ , predict the attention weights as λ varies over $\{0.1, 1, 10, 100\}$. Specifically: at which λ does the attention become $> 99\%$ concentrated on key 2?
3. **Equal keys.** If all keys are equal (so $\mathbf{k}_j$ is the same vector for every j), predict the attention distribution and the output, without computation. Justify from the definition.
4. **Gradient sanity.** Prop. 2.2 gives $\partial p_i / \partial s_j = p_i(\delta_{ij} - p_j)$. Predict which entries of the autograd-computed Jacobian will be exactly zero, positive, or negative, and check against PyTorch.
5. **Handcrafted self-attention.** The notebook sets up a 5-token toy sequence where tokens 0 and 3 share a signal. Predict which attention rows will concentrate mass on which source tokens before running the cell.

For each: direction, expected magnitude, and confidence level.